TECHNICAL RECORDS

Semantic Diffraction Gate

Architecture & Algorithm Specification

Version 0.1 | February 2026 | Working Draft | CoAgnes Framework
Authored with Claude (Anthropic) + ChatGPT (OpenAI)

License: CC BY 4.0

ABSTRACT

This document specifies the Semantic Diffraction Gate (SDG), a research architecture for multi-dimensional semantic analysis of text fragments. SDG replaces linear extraction pipelines with a parallel diffraction model in which a single structured prompt decomposes input into a maximally distributed spectrum of semantic attractors. A question-based condensation layer functions as a polarization filter, and structural integration reassembles results into traceable, auditable outputs.

The architecture preserves full provenance from source fragment through each processing stage to final projection. All algorithms are illustrated with examples from legal agreement analysis.

This document represents a working technical record rather than a finalized implementation specification. Its purpose is to articulate core architectural and algorithmic principles in a transparent, publicly inspectable form. Specific implementation choices (model selection, embedding configurations, threshold tuning, storage technologies) are intentionally left open and may evolve.

The publication of this draft is intended both as an invitation to careful technical scrutiny and as a contribution to the open discussion of structured semantic analysis methods.

1. Core Concepts and Definitions

The Semantic Diffraction Gate architecture is built on a foundational observation: when a language model processes a text fragment through a single, carefully structured prompt, the resulting outputs form a **semantic spectrum** rather than a single interpretation. This spectrum, if properly controlled, reveals the full range of meanings encoded in the original fragment.

1.1 The Gate

The Gate is the central processing unit of the architecture. It receives any text fragment, which may be a user query, a factual statement, a quotation, or an informational description, and passes it through a diffraction prompt that decomposes it into multiple semantic attractors in a single operation.

The key architectural constraint is the **one-to-many principle**: one prompt invocation must produce multiple orthogonal fragments. This distinguishes SDG from linear pipelines where fragments are extracted sequentially, each conditioned on previous extractions.

1.2 Semantic Attractors

An attractor is a discrete semantic fragment extracted from the input through the diffraction process. Each attractor represents a projection of the input along a specific semantic axis. The set of all attractors constitutes the **diffraction spectrum** of the input.

Attractors must satisfy three invariants:

- **Grounding**
  Every attractor traces to specific elements of the source text.

- **Orthogonality**
  Minimum pairwise semantic distance exceeds a configurable threshold.

- **Conservation**
  The aggregate information content of all attractors is commensurate with the source, neither losing nor fabricating information.

1.3 Differentiation from Linear Pipelines

In a sequential extraction pipeline, each step operates within the semantic corridor established by preceding steps. This creates path dependency: the model converges toward a locally-averaged interpretation rather than revealing the full spectral content of the input.

The parallel diffraction model forces the model to **distribute its attention across the entire semantic space** simultaneously, yielding a balanced matrix spectrum where each fragment is maximally distant from others.

 License: CC BY 4.0

1.3 Legal Example: NDA Clause Analysis

Input: A mutual non-disclosure clause from a commercial agreement.

In a sequential extraction workflow, the model may proceed as follows:

1. Extract primary obligation: confidentiality duties.
2. Extract duration of confidentiality.
3. Extract enumerated exceptions.

Each step narrows the semantic corridor. Subsequent extractions are conditioned on prior outputs, increasing the probability that interpretation converges around the initially identified dominant theme (confidentiality scope).

Such pipelines often emphasize explicit content while under-representing secondary structural dimensions unless explicitly prompted.

SDG Diffraction

Under SDG, a single diffraction prompt distributes attention across multiple semantic axes simultaneously. The resulting attractor spectrum may include:

- **Scope Definition**
  The definitional boundaries of “Confidential Information,” including inclusions and exclusions explicitly stated in the clause.

- **Distribution of Obligations**
  The allocation of duties between parties, including any asymmetry in wording (e.g., differences in standard of care, disclosure permissions, or exceptions).

- **Enforceability Conditions (Textually Grounded)**
  Conditions embedded in the clause that may affect enforceability or interpretation (e.g., references to “applicable law,” “reasonable efforts,” or undefined standards). These are not assertions about a specific jurisdiction, but grounded textual elements whose practical effect would depend on governing-law context.

- **Modality Tension**
  Potential internal tension between formulations such as “reasonable efforts” and absolute prohibitions (e.g., “shall not disclose under any circumstances”), where both appear within the same clause.

- **Temporal Interaction**
  The interaction between confidentiality duration and termination or survival provisions, where present in the clause.

 License: CC BY 4.0

- **Exception Structure and Cross-Reference Risk**
  The structure of carve-outs (e.g., publicly available information, independently developed knowledge) and their potential overlap or ambiguity within the clause itself.

Clarification on grounding

Each attractor must satisfy grounding invariants:

- All semantic claims are traceable to explicit spans in the clause.
- Derived tensions (e.g., modality conflict) are supported by directly cited textual formulations.
- No external assumptions (e.g., specific jurisdictional rules) are introduced unless explicitly referenced in the clause.

Sequential vs. Diffraction Approaches

The purpose of this example is not to claim that human legal analysis would overlook dimensions such as asymmetry or modality tension. Rather, the contrast highlights a common behavior in automated sequential LLM pipelines: path-dependent narrowing around the first extracted theme.

The SDG diffraction model mitigates this by forcing simultaneous multi-axis extraction, increasing the likelihood that structural dimensions — definitional, distributive, modal, temporal, and exception-based — are surfaced in a single operation.

 License: CC BY 4.0

2. Diffraction Layer

The diffraction layer is responsible for transforming an input fragment into a distributed semantic spectrum. This transformation is governed by a diffraction grid, a structured set of semantic axes that define independent perspectives along which meaning can be projected.

2.1 Orthogonality Principles

Axes in SDG are not treated as ad-hoc extraction categories or domain-specific heuristics. Instead, they function as **irreducible projection principles**: each axis represents a dimension of meaning that cannot be reconstructed as a linear combination of others without loss of semantic structure.

The core requirement of the diffraction grid is **orthogonality**. Orthogonality here is not metaphorical but operational: projections along different axes must yield attractors that are maximally independent in semantic space while remaining grounded in the same source fragment. This constraint ensures that diffraction distributes attention across structurally distinct dimensions of meaning rather than fragmenting the text arbitrarily.

The diffraction grid thus serves as the semantic equivalent of a grating in physical diffraction: it determines which structural “frequencies” of meaning are resolved and prevents premature convergence toward a single interpretive frame.

2.2 Universal Axes

The first diffraction phase applies a set of **universal axes**. These axes are invariant across domains and are intentionally not derived from the input fragment itself. Instead, they define stable perspectives that must remain available regardless of topic, style, or domain specificity.

Universal axes in SDG should be understood as a minimal operational basis rather than a claim of ontological completeness. They are selected to satisfy independence, coverage, and stability requirements across domains, providing a reproducible starting point for semantic diffraction.

The current SDG specification defines the following universal axes as a minimal mandatory set:

- **Factual Content**
  What the fragment explicitly states, independent of interpretation.
- **Presuppositions**
  What must be true for the fragment to be meaningful or operative.
- **Intentional Layer**
  The apparent purpose, objective, or communicative agenda embedded in the fragment.
- **Boundary Conditions**
  Explicit or implicit limits on applicability, scope, or validity.
- **External Dependencies**
  References or implicit links to other documents, domains, rules, or systems.

 License: CC BY 4.0

- **Internal Tensions**
  Ambiguities, contradictions, or modality conflicts present within the fragment itself.

These axes are designed to span **structurally independent dimensions of meaning**, ensuring that no single axis dominates the diffraction process. Their role is not to exhaustively classify content but to guarantee broad semantic dispersion during the initial pass.

2.3 Adaptive Axes

While universal axes provide broad spectral coverage, they are intentionally coarse. Following the initial diffraction pass, the system analyzes the resulting attractor spectrum to identify regions of high semantic density, unresolved tension, or structurally salient patterns.

From this analysis, the system generates **adaptive axes**: domain- or fragment-specific projection principles that refine resolution where it is most informative. This two-phase approach mirrors spectroscopy practice: a coarse grating is used to survey the spectrum, followed by a finer grating to resolve significant spectral lines.

Adaptive axes are not arbitrary refinements. They must satisfy the same orthogonality and grounding constraints as universal axes and are introduced only where they demonstrably increase semantic coverage or structural clarity.

Legal example:
In a force majeure clause, universal diffraction may reveal an internal tension between traditional formulations (“acts of God”) and modern enumerations (cyber incidents, pandemics). Adaptive axes may then be generated to explore dimensions such as burden of proof allocation, interaction with insurance obligations, notification requirements, or partial versus total excuse of performance.

2.4 Orthogonality and Dispersion

To prevent redundancy and semantic collapse, the attractor set is evaluated for orthogonality. Orthogonality in SDG is defined at the semantic-structural level: attractors are considered orthogonal if they project the source fragment along distinct semantic aspects that cannot be reduced to one another without loss of meaning.

This structural orthogonality is assessed primarily through comparison of semantic signatures (actor, modality, action, object, conditions, scope) and axis assignment. Attractors whose core semantic roles coincide are not considered orthogonal, even if they differ lexically.

Embedding-based similarity is used strictly as a supporting diagnostic signal. Cosine distance between embeddings computed over normalized semantic signatures and labels serves to detect near-duplicates, excessive semantic overlap, or unintended collapse of projections. Embedding metrics do not override structural distinctions and never serve as the sole criterion for orthogonality decisions.

 License: CC BY 4.0

To operationalize orthogonality control, SDG employs an adaptive sensitivity mechanism. While semantic orthogonality is defined structurally, numeric thresholds are used to tune the strictness of redundancy detection based on input complexity.

A lightweight complexity estimation pre-pass evaluates fragment length, entity density, and syntactic clause depth to produce a scalar complexity score $c \in [0, 1]$.

One possible policy for adjusting geometric sensitivity is:
threshold_ortho = base_threshold × (1 − 0.3 × c)

This formulation is illustrative rather than prescriptive. It encodes the principle that denser or more heterogeneous fragments naturally admit closer semantic projections, while short or tightly scoped inputs require higher dispersion.

Alternative thresholding strategies may be substituted, provided they preserve structural orthogonality, grounding constraints, and downstream coherence.

2.5 Coverage and Refinement

A lightweight concept map **C(F)** is constructed, representing key entities, obligations, terms, and structural components present in the fragment. Semantic coverage is computed as:

coverage = (number of concept nodes referenced by ≥1 attractor) / (total number of concept nodes)

If coverage falls below a configurable threshold:

- uncovered concept nodes or structural regions are identified;
- targeted extraction instructions are generated;
- a second focused diffraction pass is executed;
- results are merged into the attractor set and revalidated.

This refinement process increases semantic completeness without reintroducing sequential path dependency.

2.6 Conservation and Coherence

Throughout diffraction, the **conservation principle** is enforced: attractors must neither introduce information absent from the source nor omit structurally significant content. Parasitic or hallucinated attractors are flagged through grounding and entailment checks.

The orthogonality principles established at the diffraction layer are preserved downstream. Question polarization and structural integration operate over the same distributed semantic space, ensuring that subsequent processing stages refine and assemble meaning without collapsing the spectral structure produced at the Gate.

 License: CC BY 4.0

2.7 Algorithm: Diffraction Extraction

D1. Receive input fragment F and context metadata M.

D2. Construct a structured extraction prompt P(F, M, axes_universal), where axes_universal define general conceptual perspectives (e.g., actors, obligations, constraints, processes, conditions).

D3. Execute P in a single LLM call → obtain an initial attractor set A = {a_1, ..., a_n}.

Each attractor a_i must include:

- label (short canonical formulation),
- axis (conceptual perspective),
- evidence_spans (explicit spans from F),
- short_description (1–3 sentence clarification),
- semantic_signature {
  actor,
  modality, // e.g., shall, may, must not
  action,
  object,
  conditions,
  scope
  }

D4. Validate grounding and structural consistency:

For each a_i:

- verify evidence_spans are present in F;
- verify semantic_signature elements are traceable to F;
- check internal consistency between label, description, signature, and evidence.

Attractors failing grounding are:

- either discarded,
- or retained with reduced confidence.

 License: CC BY 4.0

D5. Perform semantic deduplication with safeguards:

- Compute embeddings over normalized semantic_signature + label.
- For each pair (a_i, a_j), compute cosine similarity.

If similarity > threshold_dedup:

- merge attractors ONLY IF core fields match:
  actor,
  modality,
  action,
  object.
- otherwise retain both and record a "related_but_distinct" link, with explicit differentiation note.

D6. Construct a lightweight concept map C(F), representing key entities, obligations, terms, and structural components in F (C(F) may be derived via NLP extraction, LLM-assisted concept listing, or hybrid methods).

D7. Compute semantic coverage score:

coverage = (# concept nodes in C(F) referenced by ≥1 attractor) /
(total # concept nodes in C(F)).

D8. If coverage < threshold_coverage:

- Identify uncovered concept nodes or structural regions.
- Generate targeted extraction instructions focused on uncovered areas.
- Execute a second focused LLM pass.
- Merge results into A and repeat D4–D7.

D9. Emit final attractor set A* with metadata: { label, axis, evidence_spans, semantic_signature, confidence, embedding, links_to_concepts, differentiation_links (if any) }.

 License: CC BY 4.0

3. Question Layer: Polarization and Condensation

3.1 Dual-Source Question Genesis

Questions in the SDG architecture arise from two independent sources that together create an interference pattern between what the text reveals and what the task requires.

Teleological questions derive from the task context: the user's goal, the domain requirements, the regulatory framework. These are directed polarizers that select for task-relevant semantic frequencies.

Endogenous questions are generated from the attractor spectrum itself. Each attractor may raise follow-up questions that probe its implications, test its boundaries, or explore its connections to other attractors. These are exploratory probes that prevent premature narrowing.

3.2 Question Typology Example

Type	Filter Behavior	Legal Example
Fact-Question	Narrowband filter; passes only point frequencies	*What is the exact notice period specified for breach?*
Relation-Question	Bandpass filter; passes a range of connected meanings	*How does the indemnification clause interact with the limitation of liability?*
Meta-Question	Overtone filter; extracts implications rather than content	*Why does this agreement separate IP assignment from licensing?*
Counter-Question	Inverse filter; reveals what the spectrum lacks	*Under what conditions would this arbitration clause be unenforceable?*

 License: CC BY 4.0

3.3 Algorithm: Question-Based Filtering

Q1. Generate teleological questions Q_t from task context T.

Q2. Generate endogenous questions Q_e from attractor set A^*.

Q3. Merge into question set $Q = Q_t \cup Q_e$, deduplicating by semantic similarity.

Q4. Classify each $q \in Q$ by type: Fact, Relation, Meta, or Counter.

Q5. Verify typological balance: if any type is missing, generate at least one question of that type.

Q6. For each q, compute resonance score $r(q, a_i)$ for every attractor $a_i \in A^*$.

Q7. For each q, assemble micro-context C_q from attractors where $r(q, a_i) >$ threshold_resonance.

Q8. Generate answer ans_q using $LLM(q, C_q)$ — answer is grounded in spectrum only.

Q9. Attractors with no resonance to any question are tagged 'non-focal' and retained in shadow.

Q10. Emit answer set $\{(q, ans_q, C_q, type_q)\}$ with full provenance links.

Legal Example: Filtering a Licensing Agreement Spectrum

Task: Assess risk for the licensee.

Teleological question (Relation): 'What obligations survive termination?'

Attractors that resonate: survival clause attractor, IP assignment attractor, audit rights attractor.

Non-resonant but preserved in shadow: attractor about governing law choice (irrelevant to survival analysis, but critical if reactivated for jurisdictional risk assessment).

License: CC BY 4.0

4. Integration Layer: Structural Assembly

4.1 Structural Integration vs. Vector Proximity

The integration layer assembles question answers into a coherent result. Critically, this is **not** performed through embedding similarity (which would reintroduce the averaging problem). Instead, the integration prompt defines a **structural template** with named slots, and each slot is filled by the answer fragment that best satisfies its structural requirements.

The analogy is masonry rather than clustering: each stone is placed not because it resembles the others, but because it fits a specific position in the wall.

4.2 Three-Stage Integration

4.2.1 Stage 1: Thematic Clustering

Answers are grouped by thematic affinity using embedding-based clustering. This is the only stage where vector proximity is used, and it serves only to identify related answer groups, not to select or rank them.

4.2.2 Stage 2: Intra-Cluster Synthesis

Within each cluster, a synthesis prompt consolidates answers into a single coherent statement. Critically, internal tensions and contradictions are **preserved, not resolved**. The synthesis marks contradictions with explicit conflict tags, identifying the conflict type: logical (A and not-A), contextual (true in context X, false in context Y), or perspectival (valid from viewpoint P, invalid from viewpoint Q).

4.2.3 Stage 3: Cross-Cluster Assembly

The final integration prompt receives all cluster syntheses and a structural template. It assigns each synthesis to a template slot based on structural complementarity. Unfilled slots are explicitly reported as gaps.

 License: CC BY 4.0

4.3 Algorithm: Integration

I1. Receive answer set {(q, ans_q, C_q, type_q)} from Question Layer.

I2. Embed all ans_q; cluster by thematic affinity → clusters {K_1, ..., K_m}.

I3. For each K_j, generate intra-cluster synthesis S_j preserving internal conflicts.

I4. Generate structural template T from integration prompt, defining slots {slot_1, ..., slot_p}.

I5. For each slot_k, select S_j with highest structural complementarity score.

I6. Detect cross-cluster conflicts between assigned syntheses; tag and preserve them.

I7. Assemble final projection P with full provenance: P → S_j → ans_q → a_i → F.

I8. Report unfilled slots and shadow attractors as metadata.

Legal Example: Integrating a Joint Venture Agreement Analysis

Cluster 1 synthesis: 'Governance rights are formally symmetric but operationally favor Party A through casting vote and information rights provisions.'

Cluster 2 synthesis: 'Financial obligations contain a contextual conflict: contribution schedules assume fixed currency, but the governing law clause permits currency substitution.'

Integration assigns Cluster 1 to 'governance' slot, Cluster 2 to 'financial risk' slot. The currency conflict is preserved as a structural finding, not smoothed away.

 License: CC BY 4.0

5. Output Structure and Reactivation

5.1 Multi-Layer Output Object

The result of an SDG cycle is not a text string but a **multi-layered semantic object** with four access levels:

- **Surface layer**
  A human-readable text projection, suitable for presentation. This is what the user sees.

- **Structural layer**
  The graph of relationships between all elements across all processing stages.

- **Spectral layer**
  The complete attractor set with embeddings, confidence scores, axis labels, and grounding spans.

- **Context layer**
  All questions used, their sources (teleological or endogenous), resonance patterns, and shadow attractors.

5.2 Reactivation Protocols

Any SDG output can serve as input to a new SDG cycle. The system supports three reactivation modes:

New-context reactivation: a new text fragment or question is introduced. Its diffraction spectrum interferes with the existing spectral layer, generating three event types: reinforcement (existing attractors gain confidence), new lines (previously invisible meanings emerge), and spectral shift (existing attractors are reinterpreted).

Shadow reactivation: a non-focal attractor from the shadow becomes the input for a dedicated SDG cycle. This generates exploration branches that may retroactively enrich the main analysis.

Iterative deepening: the surface-layer projection is fed back through the Gate with the same context. Each pass resolves finer semantic overtones, analogous to iterative deconvolution in signal processing.

Legal Example: Reactivation on Regulatory Change

Original analysis: a data processing agreement was analyzed under GDPR.

Reactivation trigger: new regulatory guidance on AI-generated data processing.

New-context reactivation produces: reinforcement of existing consent mechanism attractors, new spectral line on automated decision-making obligations not present in the original analysis, spectral shift on 'legitimate interest' attractor which now requires additional balancing tests.

The system does not discard the original analysis. Both coexist with temporal metadata, enabling compliance teams to see exactly what changed and why.

 License: CC BY 4.0

6. Knowledge Crystallization: The Semantic Graph

6.1 Node Types

The persistent knowledge structure is a typed semantic graph with five node categories: **Source Fragments** (original input texts), **Attractors** (extracted semantic projections), **Questions** (polarization filters), **Answers** (filtered and grounded responses), and **Projections** (integrated results). Each node carries metadata including creation timestamp, confidence score, originating cycle ID, and version history.

6.2 Edge Types

Six primary edge types encode relationships: **extracted-from** (attractor → source), **resonates-with** (attractor ↔ question, weighted by resonance score), **grounds** (answer → projection), **contradicts** (attractor ↔ attractor, typed by conflict class), **refines** (later attractor → earlier attractor), and **shadows** (non-focal attractor → question set that excluded it).

6.3 Symmetry Detection

Crystals are defined not only by their constituents but by their **symmetries**: repeating structural patterns. The graph is periodically analyzed for isomorphic substructures. When the same pattern of relationships recurs across different analyses, it signals a deep structural regularity that no individual attractor captures but that manifests in the topology.

Legal Example: Cross-Agreement Symmetry

After analyzing 50 SaaS agreements, the system detects a recurring substructure:
[limitation-of-liability] ↔ contradicts ↔ [indemnification-scope] ↔ refines ↔ [insurance-requirement].

This symmetry is not stated in any single agreement but emerges from the crystalline topology. It represents an industry-wide structural tension that individual clause analysis would miss.

6.4 Versioning and Persistence

The graph is append-only. Reactivation adds nodes and edges but never removes them. Conflicting interpretations coexist with temporal metadata. This preserves the full evolution of understanding and enables temporal queries: 'What did we understand about this clause before the regulatory change?'

 License: CC BY 4.0

7. Evolution: Trace-Based Learning and Recipes

7.1 Trace Capture

Every SDG cycle produces a complete trace record: input fragment, grid configuration (axes, thresholds), extracted attractors, question set with sources, resonance patterns, integration strategy, and quality assessment of the result. This trace is the raw material for evolutionary learning.

7.2 Recipe Extraction

A **recipe** is a reusable configuration pattern extracted from successful traces. It contains: an input typology descriptor (what kind of fragments it applies to), a grid configuration (which axes, what orthogonality threshold), a question generation strategy (which question types to prioritize, what teleological prompts to use), and integration parameters (template structure, conflict handling strategy).

7.3 Algorithm: Recipe Evolution

- **R1.** After each SDG cycle, evaluate result quality (human feedback + automated metrics).
- **R2.** If quality exceeds threshold_recipe, extract recipe R from the trace.
- **R3.** Match R against existing recipe library by input typology similarity.
- **R4.** If similar recipe exists: merge (update confidence, expand applicability range).
- **R5.** If no match: register as new recipe with initial confidence.
- **R6.** Periodically: apply meta-diffraction to recipe pairs to extract higher-order patterns.
- **R7.** Degrade recipes that consistently underperform (reduce priority, do not delete).

7.4 Meta-Evolution

Recipes themselves can become objects of SDG analysis. By diffracting two successful recipes through the Gate, the system extracts their common structural patterns, yielding higher-order recipes applicable to broader classes of inputs. This is generalization through semantic diffraction rather than statistical gradient descent.

Legal Example: Recipe for Regulatory Compliance Assessment

After 20 successful DORA compliance analyses, the system extracts a recipe:

Input type: regulatory compliance clause. Grid: universal axes + adaptive axes for [reporting obligations, incident response, third-party risk, testing requirements]. Questions: prioritize Counter-type ('what conditions make this non-compliant?'). Integration: template with slots for [gap analysis, remediation priority, evidence requirements].

This recipe accelerates future DORA analyses from cold start while remaining adaptable through Phase 2 axes.

 License: CC BY 4.0

8. Evaluation, Control, and Governance

8.1 Per-Layer Quality Gates

Each processing layer has its own quality gate with specific metrics and thresholds. No output passes to the next layer without satisfying its gate conditions.

Layer	Metrics	Method	Failure Action
Diffraction	Coverage, orthogonality, grounding score	Semantics, embeddings.	Re-extract with modified axes
Questions	Type balance, non-leading verification, relevance	LLM-as-judge + typology check	Generate missing types
Filtering	Shadow ratio, critical attractor retention	Resonance distribution analysis	Widen question scope
Integration	Coherence, trace completeness, conflict transparency	Structural template audit	Rerun with revised template
Output	Faithfulness, no fabrication, bias check	Source-output alignment score	Flag for human review

8.2 Confidence Calibration

Every node in the semantic graph carries a confidence score reflecting the system's certainty in that element. Confidence propagates through layers: a low-confidence attractor reduces confidence in all downstream answers and projections that depend on it. This creates a **confidence topology** that highlights areas of the analysis requiring human attention or further reactivation.

8.3 Ethical and Compliance Controls

Three classes of controls operate as sidecar processes at every layer:

Bias detection: each layer's output is checked for proportional representation of perspectives present in the source. If the diffraction spectrum contains both pro and con attractors but the integration emphasizes only one side, the bias detector flags the imbalance.

External rule compliance: domain-specific rules (e.g., FCA regulatory requirements, PCI DSS controls, DORA provisions) are encoded as constraint predicates. Each layer's output is validated against applicable predicates. Violations trigger review rather than automatic correction, preserving human authority.

Internal consistency: new results are checked against the existing knowledge graph for contradictions. Contradictions are not suppressed but are surfaced as explicit structural findings, enabling the system to track evolving interpretations.

 License: CC BY 4.0

8.4 System Health Diagnostics

The ratio of elements surviving between layers serves as a diagnostic metric. If diffraction produces 40 attractors but only 3 survive filtering, this signals either over-narrow questions or misaligned diffraction axes. Conversely, if all attractors survive, the questions may be too broad. The ideal ratio depends on context but is tracked over time to detect systematic drift in system behavior.

Legal Example: Ethics Gate in Action

Analyzing a service agreement where one party is significantly smaller than the other.

Bias detector flags: integration output emphasizes obligations of the smaller party in 8 of 10 structural slots, while the larger party's obligations appear in only 2 slots.

This does not mean the analysis is wrong (the agreement may genuinely be asymmetric), but it triggers an explicit annotation: 'Note: obligation distribution in this agreement is asymmetric (80/20). This reflects the source document, not analytical bias. Flagged for review.'

 License: CC BY 4.0

9. Human Interface Points

9.1 Intervention Model

The SDG architecture provides defined intervention points where human judgment can enter the process. Each intervention is recorded as a typed node in the knowledge graph (**human-intervention** node) with full provenance, ensuring that human contributions are traceable alongside algorithmic outputs.

Supported interventions: **attractor injection** (human adds an attractor not generated by diffraction), **attractor suppression** (human marks an attractor as irrelevant, moved to shadow with explicit human-override tag), **question injection** (human adds a question to the polarization layer), **integration override** (human reassigns a synthesis to a different structural slot), and **confidence override** (human adjusts the confidence score of any node).

9.2 Feedback Loop

Human quality assessments of final outputs feed directly into the recipe evolution system. A human rating of 'incomplete' triggers analysis of which attractors were missed, which questions were absent, or which integration slots were underfilled, creating targeted improvement signals rather than global retraining.

 License: CC BY 4.0

10. End-to-End Processing Flow

The complete SDG pipeline for a single input fragment proceeds as follows:

- **E1.** INPUT: Receive fragment F, task context T, and optional recipe hint R_h.
- **E2.** RECIPE LOOKUP: If R_h exists, load configuration. Otherwise, classify F and select best-matching recipe from library.
- **E3.** DIFFRACTION (Phase 1): Execute universal-axis diffraction → A_1.
- **E4.** QUALITY GATE 1: Validate coverage, orthogonality, grounding of A_1.
- **E5.** DIFFRACTION (Phase 2): Generate adaptive axes from A_1; extract additional attractors → A* = A_1 ∪ A_2.
- **E6.** QUALITY GATE 2: Re-validate expanded attractor set.
- **E7.** QUESTION GENESIS: Generate Q_t from T, Q_e from A*; merge and classify → Q.
- **E8.** QUALITY GATE 3: Verify typological balance and non-leading property.
- **E9.** FILTERING: Compute resonance; generate answers from micro-contexts; tag shadow attractors.
- **E10.** QUALITY GATE 4: Check shadow ratio and critical attractor retention.
- **E11.** INTEGRATION: Cluster → synthesize → assemble into structural template.
- **E12.** QUALITY GATE 5: Verify coherence, trace completeness, conflict transparency.
- **E13.** OUTPUT CONTROL: Run bias, compliance, and consistency checks.
- **E14.** EMIT: Multi-layer output object with surface projection and full provenance graph.
- **E15.** PERSIST: Add all nodes and edges to knowledge graph; update recipe library if quality threshold met.

NOTE ON STATUS

This document is a working draft capturing architectural and algorithmic decisions as of the date shown on the title page. It is intended as a technical reference for implementation and further refinement. Specific implementation details including prompt templates, threshold values, embedding model selection, and graph storage technology are to be specified in companion documents.

License: CC BY 4.0

11. Technical notes

11.1 Roles of embeddings

Embeddings in SDG are used strictly as geometric diagnostics, not as semantic authorities. They serve three auxiliary functions:

1. Detecting excessive semantic overlap (deduplication).
2. Estimating dispersion of the attractor spectrum (orthogonality control).
3. Identifying thematic affinity prior to structural synthesis.

Embeddings never:

- determine structural slot assignment,
- override grounding constraints,
- replace explicit semantic_signature comparison.

Structural fields have precedence over vector similarity.

 License: CC BY 4.0